LAN Collaborative Whiteboard

Computer Networks Lab Project | Technical report and user guide

Project scope

The application provides a shared desktop canvas and group chat on a local network. A standalone TCP server coordinates rooms, permissions and board state. Standalone Windows clients provide drawing tools, participant indicators, chat and session controls. Operation does not require internet access or a Python installation on demo machines.

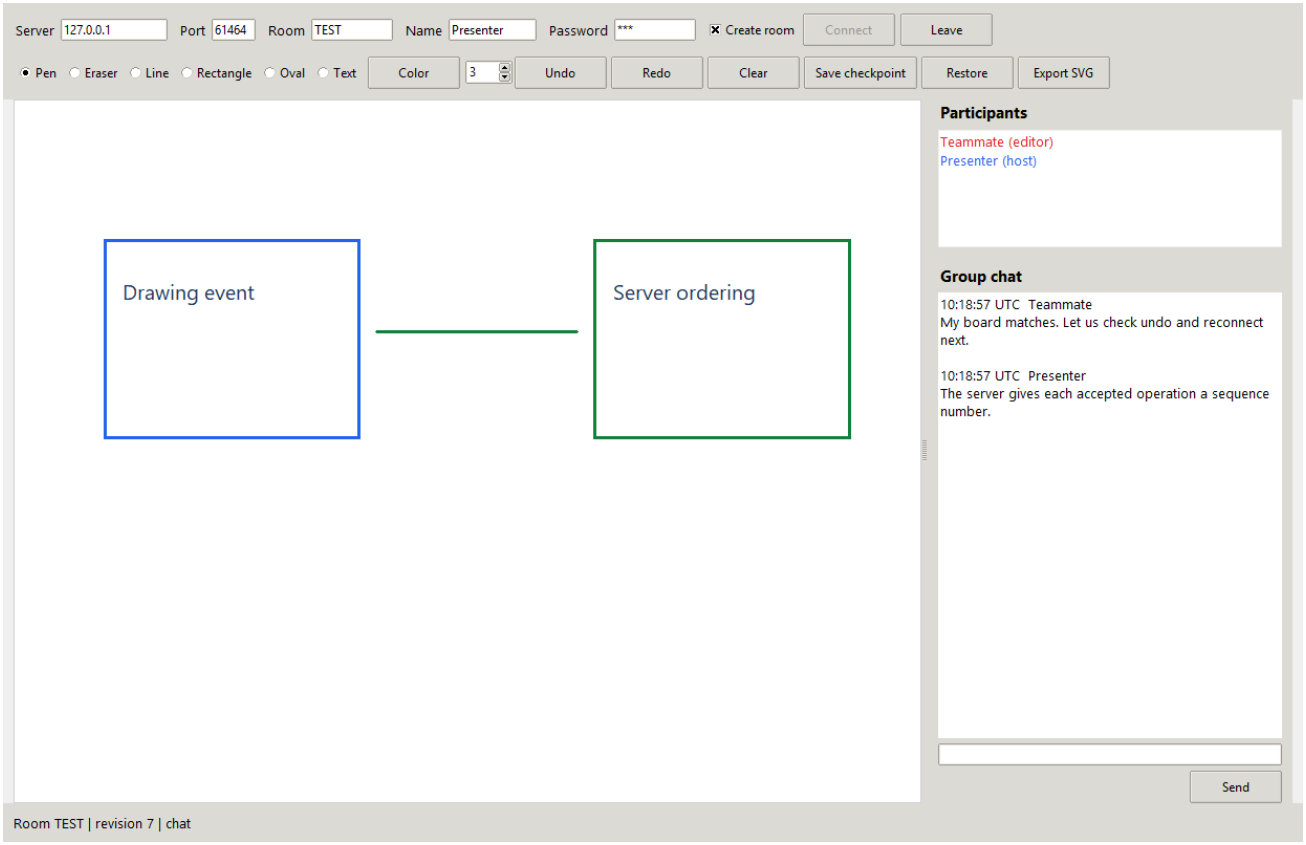

Figure 1. Running desktop client connected to a second socket participant. Both participants share the server-ordered board and chat.

Submission contents

Source code, server and client EXEs, this report and the presentation are included. The actual members must add their names, roll numbers and explanation recording before final submission. The assignment gives 15 November 2026, 11:55 pm IST as the deadline.

Architecture and socket protocol

Client-server arrangement

Client A <-- TCP/IP LAN --> Central server <-- TCP/IP LAN --> Client B. Every client connects directly to the server. Each room has one ordered board, chat history, member identities and a host. Rooms do not share drawing or chat events.

The server uses one reader thread and one queued writer per connection. A reentrant server lock covers room changes, revision assignment, persistence and broadcast enqueueing. Each client receives the same order of accepted events. Socket writes happen in writer threads so a slow recipient does not perform network writes while the state lock is held.

Frame layout

Bytes	Meaning
0-3	Unsigned 32-bit big-endian length N of the JSON payload
4 to N+3	N bytes of UTF-8 JSON; maximum N is 8 MiB

TCP is a reliable ordered byte stream, so application boundaries need explicit framing. exact() loops until the requested byte count arrives. It handles both fragmented writes and several frames arriving together. A malformed length, invalid JSON, EOF or timeout ends that connection. The client reconnects and requests current state. Corrupted framing is handled by disconnecting, rather than scanning arbitrary bytes for a new boundary.

Message	Important fields / purpose
join	room, name, password, create, token; first frame on a connection
welcome / state	Identity token and role; then seq, ops, chat and members snapshot
draw	request plus op: tool, points, color, width, text, group
chat	request and text; server adds username and UTC timestamp
undo / redo	request; server chooses that user's operation group
clear / save / restore	Host-only shared board/checkpoint actions
event / ack / error	Ordered accepted update; duplicate acknowledgement; rejected request
sync / presence / cursor	Snapshot request; membership notice; ephemeral named cursor
ping / pong	Heartbeat, sent every 5 seconds; server read timeout is 20 seconds

Example drawing request:
{"type":"draw","request":"a1","op":{"tool":"pen","points":[[100,200],[150,240]],"color":"#123456","width":3,"group":"drag1"}}. Coordinates are integer values from 0 to 10000, independent of window dimensions. The server supplies the authenticated user identity; it does not trust a client-supplied owner.

Ordering, history and recovery

A single accepted order

Each accepted persistent action increments the room revision (seq). The server broadcasts it to all room members, including the sender. Clients render the accepted order instead of permanently applying local predictions. A client discards older revisions; a gap causes a fresh state request. Join snapshots are enqueued under the same server lock as event broadcasts, so an edit cannot fall between taking a snapshot and subscribing a new client.

Each action carries a unique request ID. The server remembers the last 20,000 accepted IDs per identity. On reconnect, unacknowledged requests are retried; previously committed requests receive an acknowledgement without another state change. Deduplication is bounded and is not an unlimited exactly-once guarantee. The client pauses new edits while disconnected.

Undo and redo under concurrent drawing

Every drag has a group ID. Pen and eraser points are transmitted in short chunks of up to eight points during movement, sharing the same group. Undo disables all segments of that user's latest active group; redo restores the most recently undone eligible group. A new drawing invalidates only that user's redo branch. Neither operation removes another participant's work. Clear removes the current operations and their undo history. Restoring a checkpoint restores the saved operation list and active flags.

The eraser is a white stroke on the white background. Replaying all active operations in their original order defines its behavior when drawings overlap. Shapes commit when the mouse is released; annotations commit after text entry. Cursor messages are temporary and do not enter the revision log or saved canvas.

Persistence and connection recovery

After each accepted action, the server writes its rooms to a temporary JSON file and replaces sessions.json. This retains users, password hashes, recovery tokens, operations, deduplication IDs, chat and a single checkpoint per room. Replacing the file avoids exposing a partly written JSON document during ordinary interruption; sudden power-loss durability is not guaranteed because no fsync is issued. Use a writable local directory and keep backups of the state file.

A disconnected client retries every two seconds. Its token is stored in .cn_whiteboard_tokens.json in the user's home directory. The same token restores the identity and host role after reconnection or server restart. A replacement connection for that identity closes the old connection. Other participants continue while a client leaves or reconnects.

Access control and validation

Rooms require their code and password. The password is stored using PBKDF2-HMAC-SHA256 with a random salt and 100,000 iterations. Recovery tokens are random bearer secrets. Only the room host can clear, save or restore checkpoints. The server validates tools, colors, coordinates, widths, point counts, text sizes and frame sizes. An invalid editing request produces an error without changing the board.

This is basic access control for a trusted private LAN. Frames are not encrypted. Host ownership remains attached to its token; there is no automatic host election. Do not expose the TCP port to the public internet.

User guide and lab demonstration

Server and LAN configuration

1. Extract the ZIP to a writable folder on the server computer. Run release/WhiteboardServer.exe. The default bind address is 0.0.0.0 and the TCP port is 5050. Use --port 6060 or --storage path/to/sessions.json to change them.
2. Find the server's LAN IPv4 address using ipconfig. Allow the server on the Windows private-network firewall. Devices must be on the same reachable LAN; guest Wi-Fi or access-point isolation can block connections.
3. Run a client on the server machine using 127.0.0.1. Enter a room code of 3-24 letters, digits, hyphens or underscores, a username and a password. Select Create room once. That identity becomes host.
4. Other participants enter the server's LAN address, same port, room and password, with Create room unchecked. No internet services are contacted.

Using the workspace

Choose Pen, Eraser, Line, Rectangle, Oval or Text. Color and width apply to subsequent work. Use the participant list and named cursors to identify users. Enter a chat message and press Enter or Send. Chat records are in server order and show usernames and UTC timestamps. New users receive the current board and retained chat. Export SVG saves an image locally; Save checkpoint stores a shared server checkpoint. Restore and Clear ask for confirmation and require the host role.

A repeatable in-person demo

1. Keep the LAN active and disable internet/WAN connectivity. Start the server and connect two physical computers.
2. Draw different colored strokes at the same time and exchange chat messages. Compare board contents.
3. Undo and redo one participant's stroke while the other draws. Confirm the other participant's work remains.
4. Join a third client and confirm it immediately gets the existing board and chat.
5. As host, save a checkpoint, clear, then restore. As editor, try Clear and confirm rejection.
6. Disconnect one client's LAN connection, make another edit from a connected client, and reconnect. Confirm the recovered canvas matches.
7. Restart the server with the same state file. Clients should reconnect and recover their identities.

Troubleshooting

Connection refused: check the server process, port and host address. Timeout: check private-network firewall rules, subnet routing and Wi-Fi isolation. Wrong room: create it once, then join. Host actions rejected: use the original host computer/account and its saved token. State file errors: move the project to a writable folder; do not edit sessions.json while the server runs. If a board reaches 1000 segments, export, save a checkpoint and clear it. Each new checkpoint replaces the previous one.

Explanation video

Each actual member must explain their own contribution with name and face visible, using the slides and source. A live app demonstration is not required in that video. All members must attend the separate in-person evaluation. TEAM_RECORDING.md provides a speaking outline. Add the recording and actual team details before final submission; one member submits the ZIP.

Requirement coverage and validation

Assignment requirement	Implementation / check
Freehand, erase, color, shared consistent canvas	client.py tools; server-ordered event stream
Real-time sync, validation, concurrent edits	Server lock + per-client writer queues; four simultaneous socket writers tested
Permissioned reset, saving and restoring	Host-only clear/save/restore; disk-backed rooms and checkpoint test
Create/join by LAN address and room	Connect form and join handshake; room code/password validation
Participants, usernames, user indicators	Membership broadcasts, join/leave notices, named colored cursors
Late joining and continuity	Locked snapshot before subsequent events; late-join socket test
Transmission, broadcast, order, duplicate/missing handling	Framed TCP; seq checks; request deduplication; snapshot recovery
Disconnect, timeout and reconnect	5-second heartbeats, 20-second read timeout, 2-second client retry
Group chat, broadcasting, chronological history	Server-stamped name and UTC time; last 500 messages persist
Stroke representation, state recovery, undo/redo	Normalized operation list, identity tokens, per-user grouped undo
Standalone server/client, source comments	Two Windows x64 one-file EXEs; commented Python source
Technical PDF and user guide	This report: protocol, concurrency, recovery, setup and controls
PPT, screenshots, workflow and examples	Editable presentation with captured application windows and shared-board examples
Member explanation video and final ZIP	Team must record the video; prepared archive includes other deliverables
Offline live evaluation and team participation	Procedure on previous page; physical LAN rehearsal remains a team task
Access control, platform and performance	Private LAN, Windows EXEs, room password + host role, bounded queues
Additional features	Shapes, text annotations and SVG export

Checks completed

Ten automated tests passed: stream fragmentation and coalescing, malformed and truncated frames, Unicode message framing, join/chat/permissions/checkpoint, duplicate requests and per-user undo, four concurrent writers with identical revisions, reconnect and server restart recovery, wrong password and oversized frame rejection, room isolation and membership changes, all drawing tools and redo-branch behavior. A Tk GUI integration check also passed drawing, erasing, shape and text rendering, chat, undo/redo, checkpoint controls, SVG export and automatic reconnect. These checks used this Windows computer and local TCP sockets. They do not measure physical LAN latency or verify a second computer's firewall.

Limits and future work

The application caps connections at 64 and board segments at 1000. State is serialized to disk for every accepted persistent event, so large deployments require a journal or database. Further work could benchmark LAN latency, add TLS and support host transfer. No physical LAN performance measurements are claimed.